

- This handler is synchronized with the GUI thread when executed, so you can safely modify UI elements.
- `onPostExecute` When `doInBackground` has completed, the return value from that method is passed to this event handler.
- Use this handler to update the UI once your asynchronous task has completed. `onPostExecute` is synchronized with the GUI thread when executed, so you can safely modify UI elements.

Running an Asynchronous Task

Once you've implemented your asynchronous task, execute it by creating a new instance and by calling `execute`. You can pass in a number of parameters, each of the type specified in your implementation.

```
new MyAsyncTask().execute("inputString1", "inputString2");
```

5.5.2 Manual Thread Creation and GUI Thread Synchronization

While `AsyncTask` is a useful shortcut, there are times when you'll want to create and manage your own `Threads` to perform background processing. In this section you'll learn how to create and start new `Thread` objects, and how to synchronize with the GUI thread before updating the UI.

Creating a New Thread

You can create and manage child threads using Android's `Handler` class and the threading classes available within `java.lang.Thread`. The following code is for moving processing onto a child thread.

```
// This method is called on the main GUI thread.
private void mainProcessing() {
    // This moves the time consuming operation to a child
    thread.
Thread thread = new Thread(null, doBackgroundThreadProcessing,
                           "Background");

    thread.start();
}

// Runnable that executes the background processing method.
private Runnable doBackgroundThreadProcessing = new Runnable() {
    public void run() {
        backgroundThreadProcessing();
    }
};
```